Intro

Welcome to Bliss Applications **iOS** Challenge!



Image taken from here

In this exercise you'll be putting your skills to the test by creating a simple and fun application. For that, you'll have to use your favorite IDE (XCode / AppCode), create a *github* account, share the url and, if private, give us access to the project (we'll provide a user).

That being said, let's jump into the next page where you will find some considerations and tips.

Considerations

Layouts and Navigation:

This app will have the following structure:



Note: This is just a representative approach to give you a bigger picture of the final result and help you with the architecture planning. UI and UX won't be targets of evaluation.

API:

For unauthenticated requests, the rate limit allows for up to 60 requests per hour. Unauthenticated requests are associated with the originating IP address, and not the user making requests.

Tip: after solving the first question you are able to mostly use cache hits, in other words, you are able to solve all the questions with the exception of the last one which is a paginated data consumption (that can also be cacheable but let's forget that ^^)

Endpoints:

1. GET https:*//api.github.com/emojis*
2. GET https:*//api.github.com/users/:username*
3. GET https:*//api.github.com/users/:username/repos (parameters: int page, int size)*

GIT:

At Bliss we use GIT as a version control system. We also like Git Flow which we consider, like the author, a successful Git branching model. It's not mandatory for this test since you're the only developer in it, but we'll take it as a bonus ;)

Language:

Swift.

Final thoughts:

The questions order are made with an architectural point of view in mind, in other words, if you want to take advantage of reutilization, we advise you to give a quick look but don't skip tasks until you've completed question number 1, 2 and 3.
Good luck!

Challenge

1. Everyone likes emojis right? So let's grab some from Github's API. In this first task you'll have to fetch all emojis.

   Create a simple **"Get Emoji"** button to invoke that call.

   Use **endpoint 1,** and as a response you'll receive a gigantic *Map<String, String>*.
   ex:
   ```
   {
     "+1": "https://github.githubassets.com/images/icons/emoji/unicode/1f44d.png?v8",
     "-1": "https://github.githubassets.com/images/icons/emoji/unicode/1f44e.png?v8",
     "100": "https://github.githubassets.com/images/icons/emoji/unicode/1f4af.png?v8",
     "1234": "https://github.githubassets.com/images/icons/emoji/unicode/1f522.png?v8",
     "1st_place_medal": "https://github.githubassets.com/images/icons/emoji/unicode/1f947.png?v8",
     "2nd_place_medal": "https://github.githubassets.com/images/icons/emoji/unicode/1f948.png?v8",
     ...
   }
   ```

   Seriously github?! Couldn't you at least send us a *List<Emoji>* 🤔
   ex:
   ```
   [
     {"name":"+1", "url":"https://github.githubassets.com/images/icons/emoji/unicode/1f44d.png?v8"},
     ...
   ]
   ```

   Well, looks like we'll have to use a typealias for that one, but no worries, It's simpler than its looks, just keep in mind that the object that came from the API is a dictionary having a String as its key and value.

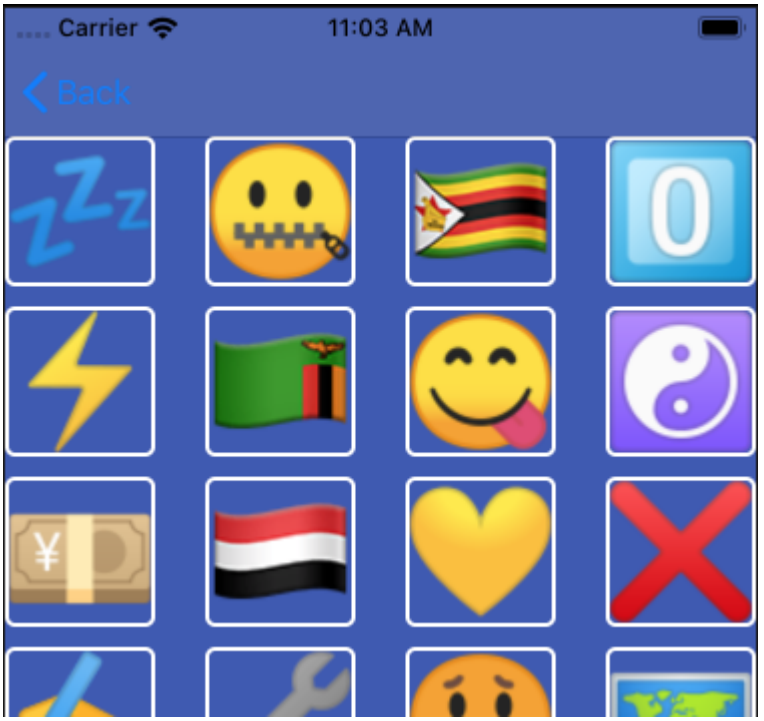
2. Time to persist this data, how it's up to you, but *Core Data* is a good idea! Feel free to use any of your favourite approaches.

3. Modify your fetch logic to only fetch from API if there's no cached data. Remember, this is very important since API calls are limited.

4. Now that we have some data to play with let's do exactly that, change your **"Get Emoji"** button to **"Random Emoji"** and everytime you click it display a random emoji:



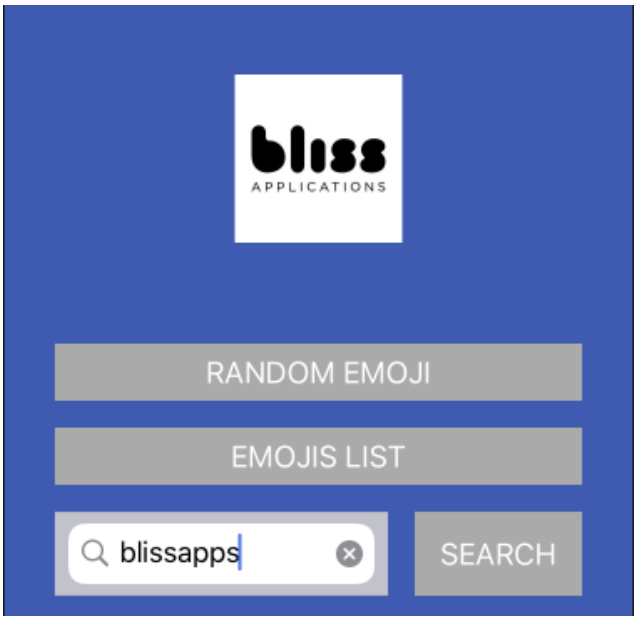
5. Add another button, **“Emoji List”**, to list all the stored emojis. Make this button open a new screen displaying all the items in a grid:



6. Let’s make this list more dynamic by removing an item when clicked. **Note (1): remove it only from memory not from your cache. (2): Your list should keep the same order after the deletion of that item.**

   Bonus: add a *Pull-To-Refresh* behaviour to reset the list by listing all the Emojis back again.

7. Your next feature will be searching for user avatars by name. Create an input field and a search button:



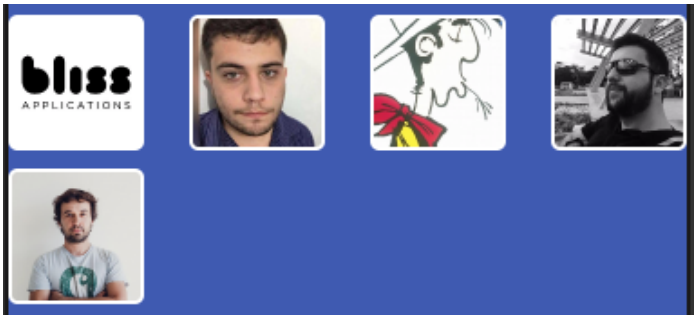
Use **endpoint 2,** and as a response you'll receive:

```
ex: {
  "login":"blissapps",
   "id":223156,
   "avatar_url":"https://avatars0.githubusercontent.com/u/223156?v=4",
}
```

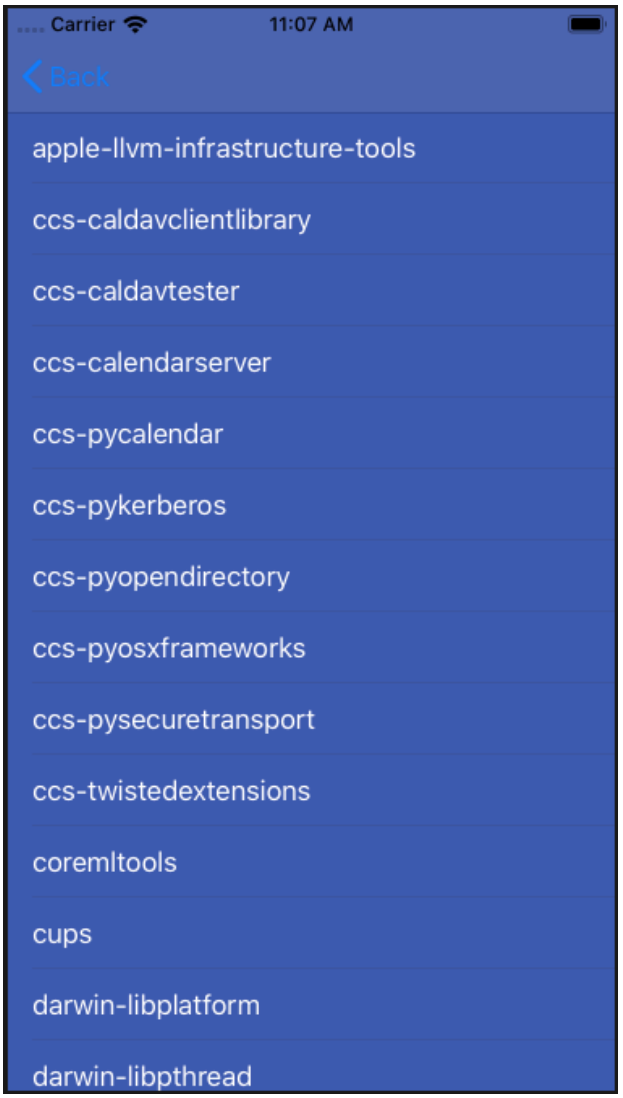
Query the API for that user and retrieve its image. Store it in your DB, you'll need it soon.

Bonus: implement cache hits, in other words, check your DB before going for the API, just like in question 3.

8. Time for a new button! "**Avatar List"** is a great name, implement the same logic as in question 5 with a "small" difference: Delete operations will remove the item from the DB and no *Pull-To-Refresh* will be needed:



9. Final task in this quest! Add a last button **“Apple Repos”** and list all Apple’s GitHub repos. This should be implemented with pagination (fetch 10 repositories per request, if the user scrolls until the end of the list fetch for more 10):



Use **endpoint 3,** and as a response you’ll receive:

```
ex: [
    {
    "id":170908616,
    "full_name":"apple/.github",
    "private":false,
    },
    ...
]
```

Congratulations!

By reaching this last page we truly hope you've fully completed this test. Review your code, all your *commits* and *repo*, and make sure you've *pushed* your final version. There's only one little task left:

- Merge it with *master* and create a *tag* with the name *ready_for_evalution*;
- Send us an email to recruitment-team@blissapplications.com

Fingers crossed - now it's our turn -, we'll keep in touch, stay tuned and thanks for your effort.

